Neuro-Adaptive Control Without State Measurement: A 1D-CNN Framework for Higher-Order Systems Using Historical Data

Myeongseok Ryu, Naol Samuel Erega, and Kyunghwan Choi

***Abstract*—This study proposes a convolutional neural network (CNN)-based adaptive controller with three notable features: 1) it determines control input directly from historical sensor data; 2) it learns the desired control policy during real-time implementation without using a pre-trained network; and 3) the asymptotic tracking error convergence is proven during the learning process. An adaptive law for learning the desired control policy is derived using the gradient descent optimization method, and its stability is analyzed based on the Lyapunov approach. A simulation study using a control-affine nonlinear system demonstrated that the proposed controller exhibits these features, and its performance can be tuned by manipulating the design parameters. In addition, it is shown that the proposed controller has a superior tracking performance to that of a deep neural network (DNN)-based adaptive controller.**

***Index Terms*—Neuro-adaptive control, convolutional neural network (CNN), ...**

NOTATION

In this study, the following notation is used:

- $\mathbb{R}^n$ denotes the n-dimensional Euclidean space.
- $\mathbb{R}^{n\times m}$ denotes the set of $n \times m$ real matrices.
- A vector and a matrix are denoted by $\boldsymbol{x} = [x_i] \in \mathbb{R}^n$ and $\boldsymbol{A} := [a_{ij}] \in \mathbb{R}^{n\times m}$, respectively.
- $\boldsymbol{I}_n$ denotes the $n \times n$ identity matrix, and $\mathbf{0}_{n\times m}$ denotes the $n \times m$ zero matrix.
- $\mathrm{vec}(\boldsymbol{A})$ denotes the vectorization of the matrix $\boldsymbol{A}$.
- A matrix $\boldsymbol{P} \succ 0$ ($\boldsymbol{P} \succeq 0$) is positive definite (positive semidefinite).
- $\lambda_{\min}(\boldsymbol{A})$ denotes the minimum eigenvalue of the matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$.
- $\otimes$ denotes the Kronecker product [Bernstein and Bernstein, 2009].

I. INTRODUCTION

[MS: Naol I ask you to write the following things]

- The jacobian of NN is missing (appendix in previous paper)

Myeongseok Ryu, Naol Samuel Erega and Kyunghwan Choi are with the Cho Chun Shik Graduate School of Mobility, Korea Advanced Institute of Science and Technology (KAIST), Daejeon 34051, Korea (e-mail: dding_98@kaist.ac.kr, samuelnaol7@kaist.ac.kr and kh.choi@kaist.ac.kr).

[MS: Should be reviewed] This work was supported in part by a Korea Research Institute for Defense Technology planning and advancement (KRIT) grant funded by Korea government DAPA (Defense Acquisition Program Administration) (No. KRIT-CT-22-087, Synthetic Battlefield Environment based Integrated Combat Training Platform) and in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2025- 00554087). (Corresponding author: Kyunghwan Choi)

- can you change notations of NN? (go to above part where newcommands are defined)
- I removed some notations in your explanation about test circuit. If you think they are necessary, please let me know or if not, re-draw the figure.
- Think about controllers to compare with for comparative study
- Explain how you implemented the simulation

A. Motivation

Neural networks (NNs) have been widely used in control applications as a function approximator for adaptive control [Esfandiari et al., 2022], state estimation [Talebi et al., 2010], and so on. These studies provided the ultimate boundedness of tracking or estimation errors based on the Lyapunov-based stability analysis but were limited to

Deep neural networks (DNNs) with multiple hidden layers are more expressive and can provide better performance than shallow NNs. However, due to the difficulty in deriving an adaptation law ensuring stability, designing a stable DNN-based controller is generally considered challenging. There have been a few attempts at overcoming this challenge, including developing Lyapunov-based adaptation laws for controllers based on DNNs or their variations [Patil et al., 2021], [Griffis et al., 2023], [Hart et al., 2023]. A Lyapunov-based adaptation law for a DNN-based controller was derived in [Patil et al., 2021], which ensured asymptotic convergence of tracking error. [Griffis et al., 2023] utilized long short-term memory (LSTM), a type of DNN, in designing an adaptive controller and presented the ultimate boundedness of tracking error based on Lyapunov analysis. The Lyapunov-based approach was extended to using physics-informed LSTM for an adaptive controller and therein demonstrated its asymptotic stability [Hart et al., 2023].

While DNN-based adaptive control with a stability guarantee has been studied by pioneering works [Patil et al., 2021], [Griffis et al., 2023], [Hart et al., 2023], convolutional neural networks (CNNs), which are well known for their spatial feature capturing ability, have not been as actively investigated in control applications, as they have been in computer vision applications. Nonetheless, motivated by the feature-capturing capability, the use of CNNs as a basis for end-to-end controllers has been studied [Rausch et al., 2017], [Jhung et al., 2018], [Liu and Dong, 2017], [Nobahari and Seifouripour, 2019]. These end-to-end controllers determine the control input directly from the sensor data with features that are extracted by CNNs. In

[Rausch et al., 2017], [Jhung et al., 2018], CNN-based end-to-end controllers were trained offline to produce the desired steering behavior based on 2D images from the camera. [Liu and Dong, 2017], [Nobahari and Seifouripour, 2019] generated pseudo-2D images by stacking historical system input or output data and used them to train CNN-based end-to-end controllers offline to behave as target controllers. However, none of the works above considered online adaptation or stability analysis of CNN-based end-to-end controllers.

This study presents a CNN-based adaptive controller with a stability guarantee. The proposed controller uses 2D images generated by stacking historical sensory data as an input matrix to convolutional layers (CVLs). The output of CVLs is input to the following fully connected layers (FCLs), which produce the controller output. An adaptation law of network weights is implemented to learn the desired control policy and is derived using the gradient descent optimization method. The stability of the adaptation laws is proven based on the Lyapunov analysis by showing that the tracking error is asymptotically convergent and the network weights and biases are bounded. A part of the proposed controller is designed based on these findings, such as Jacobians with respect to FCLs from [Patil et al., 2021] and stabilizing techniques for the adaptation law from [Talebi et al., 2010], [Esfandiari et al., 2022]; however, the remainder of the proposed controller, particularly the components rely on the mathematical formulation and adaptation law of the overall CNN architecture, has been solely developed by the present authors. A simulation study using a control-affine nonlinear system is performed to demonstrate that the proposed CNN-based controller ensures asymptotic convergence of tracking errors during online adaptation without using a pre-trained network. In addition, the tracking performance of the proposed controller is compared with that of the DNN-based adaptive controller, which is a form in which the CVLs are not included in the proposed controller.

[MS: 1D-CNN is suitalbe for time serial data, and the computational cost is lower than DNNs!]

B. Contributions

C. Organization

The remainder of this paper is organized as follows. Section II describes the system details of the proposed CVL-CONAC architecture. Section III presents the problem formulation and control objectives. Section IV develops the adaptation laws. Section V provides the stability analysis. Section VI presents the numerical validation results. Finally, Section VII concludes the paper.

II. Problem Formulation and Control Objectives

Consider the 2nd order affine strict feedback nonlinear system:

$$\begin{cases} \frac{\mathrm{d}}{\mathrm{d}t}\boldsymbol{x}_1 = \boldsymbol{f}_1(\boldsymbol{x}_1, \boldsymbol{x}_2) \\ \frac{\mathrm{d}}{\mathrm{d}t}\boldsymbol{x}_2 = \boldsymbol{f}_2(\boldsymbol{x}_1, \boldsymbol{x}_2) + \boldsymbol{g}(\boldsymbol{x}_1, \boldsymbol{x}_2)\boldsymbol{u}, \end{cases} \tag{1}$$

where $\boldsymbol{x}_i \in \mathbb{R}^n$ and $\boldsymbol{u} \in \mathbb{R}^n$ denote the state and control input, respectively, and $\boldsymbol{f}_i : \mathbb{R}^{2n} \to \mathbb{R}^n$ and $\boldsymbol{g} : \mathbb{R}^{2n} \to \mathbb{R}^{n\times n}$

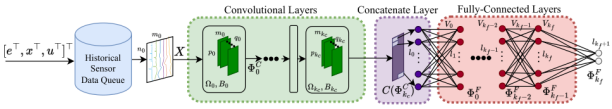

Fig. 1: System Architecture.

are unknown nonlinear functions for $i \in \{1, 2\}$. The system output is defined as $\boldsymbol{y} = \boldsymbol{x}_1$, assuming that the state $\boldsymbol{x}_1$ can be measured while $\boldsymbol{x}_2$ is not measurable.

The system (1) satisfies the following assumptions:

Assumption 1. *The control effectiveness matrix $\boldsymbol{g}(\cdot)$ is positive definite, i.e., $\lambda_{\min}(\boldsymbol{g}(\cdot)) > 0$.*

[MS: Add additional assumptions if necessary.]

We assume that the desired trajectory $\boldsymbol{y}_d(t) \in \mathbb{R}_{\leq 0} \to \mathbb{R}^n$ is given and satisfies the following assumption:

Assumption 2. *The desired trajectory $\boldsymbol{y}_d(t)$ is bounded, such that $\|\boldsymbol{y}_d\| \leq \overline{y}_d$ for some $\overline{y}_d \in \mathbb{R}_{>0}$.*

The control objective is to design a NAC using 1D-CNN framework that makes the system output $\boldsymbol{y}$ track the desired trajectory $\boldsymbol{y}_d$ by generating the control input $\boldsymbol{u}$ directly from the historical data (without state), without using a pre-trained network, and ensuring the stability of the closed-loop system during the adaptation process.

III. Proposed Controller

A. Controller Architecture

The proposed controller architecture is shown in Fig. 1. The control input $\boldsymbol{u}$ is solely determined by the NN $\widehat{\boldsymbol{\Phi}}$ without any additional terms, The NN input is generated by stacking historical data as follows:

$$\boldsymbol{Z} := \begin{bmatrix} \boldsymbol{z}(t) & \boldsymbol{z}(t-\Delta t) & \cdots & \boldsymbol{z}(t-(k_c-1)\Delta t) \end{bmatrix} \in \mathbb{R}^{n\times k_c}, \tag{2}$$

where $\boldsymbol{z} := \left(\boldsymbol{y}^\top, \boldsymbol{y}_d^\top\right)^\top \in \mathbb{R}^{2n}$, Δt is the sampling time and k_c is the number of sampled data points. The NN $\widehat{\boldsymbol{\Phi}}$ consists of convolutional layers (CVLs) and fully-connected layers (FCLs), which are detailed in the following subsections.

1) Convolutional Layers (CVLs): The CVLs are represented recursively as:

$$\boldsymbol{\Phi}^C_{j_c} = \begin{cases} O(\boldsymbol{\Phi}^C_{j_c}(\Phi^C_{j_c-1}), \Omega_{j_c}, B_{j_c}), & j_c \in [1, \ldots, k_c] \\ O(\phi^C_0(X), \Omega_0, B_0), & j_c = 0 \end{cases}, \tag{3}$$

where $O : \mathbb{R}^{n_{j_c}\times m_{j_c}} \times \mathbb{R}^{p_{j_c}\times m_{j_c}\times q_{j_c}} \times \mathbb{R}^{q_{j_c}} \to \mathbb{R}^{n_{j_c+1}\times m_{j_c+1}}$ denotes the CNN operator (see Appendix A), and $\boldsymbol{\Phi}^C_{j_c} : \mathbb{R}^{n_{j_c}\times m_{j_c}} \to \mathbb{R}^{n_{j_c}\times m_{j_c}}$ denotes the matrix activation function (i.e., $\phi^C(\Phi^C_{j_c-1})_{(i,j)} = \sigma_{j_c}(\Phi^C_{j_c-1})_{(i,j)}$) for some activation function $\sigma_{j_c} : \mathbb{R} \to \mathbb{R}$. The first activation function ϕ^C_0 should be a bounded nonlinear function to ensure that the input to the first CNN operator is bounded. In this study, $\alpha_1 \tanh(\cdot)$ is selected with $\alpha_1 \in \mathbb{R}_{>0}$. The filter set Ω_{j_c} contains q_{j_c} filters $W^{(i)}_{j_c} \in \mathbb{R}^{p_{j_c}\times m_{j_c}}$, $\forall i \in [1, \ldots, q_{j_c}]$, and is represented as $\Omega_{j_c} = \{W^{(1)}_{j_c}, \ldots, W^{(q_{j_c})}_{j_c}\}$, where superscript i denotes the filter index. The bias vector B_{j_c} consists of q_{j_c} biases.

The weights for the j_c-th CVL layer are collectively represented as $\widehat{\boldsymbol{\theta}}_{Cj_c} \triangleq [\text{vec}(\Omega_{j_c})^T, B_{j_c}^T]^T$.

2) Fully-Connected Layers (FCLs): The output matrix of the CVLs is input to the concatenate layer $C(\Phi_{k_c}^C) = [\text{vec}(\Phi_{k_c}^{C^\top})^\top, 1]^\top$ before the input layer of FCLs for compatibility between the CVLs and FCLs. The FCLs are represented recursively as:

$$\Phi_{j_f}^F = \begin{cases} V_{j_f}^\top \phi_{j_f}^F(\Phi_{j_f-1}^F), & j_f \in [1, \dots, k_f] \\ V_0^\top C(\Phi_{k_c}^C), & j_f = 0 \end{cases}, \tag{4}$$

where $V_{j_f} \in \mathbb{R}^{l_{j_f}+1 \times l_{j_f+1}}$ denotes the weight matrix, $\phi_{j_f}^F : \mathbb{R}^{l_{j_f}} \to \mathbb{R}^{l_{j_f}+1}$ denotes the vector activation function defined by $\phi_{j_f}^F(\Phi_{j_f-1}^F) \triangleq [\sigma_{j_f}(\Phi_{j_f-1}^F)^\top, 1]^\top$ for $j_f \in [1, \dots, k_f - 1]$, for some nonlinear activation functions $\sigma_{j_f} : \mathbb{R} \to \mathbb{R}$ such as $\tanh(\cdot)$. Note that $C(\Phi_{k_c}^C)$ and $\Phi_{j_f}^F$ are augmented by 1 to consider the bias of the preceding input layer as a weight in the weight matrix. The output of the FCLs is the final output and is represented as $\widehat{\boldsymbol{\Phi}} \triangleq \Phi_{k_f}^F$.

The FCL weights for the j_f-th layer are $\widehat{\boldsymbol{\theta}}_{Fj_f} \triangleq \text{vec}(V_{j_f})$.

B. Adaptation Laws Derivation

An end to end controller(i.e. $u = \widehat{\boldsymbol{\Phi}}$) is designed to drive errors defined by (5) to zero:

$$e = \begin{bmatrix} e_y \\ e_\psi \end{bmatrix} = \begin{bmatrix} y - y_{\text{des}} \\ \psi - \psi_{\text{des}} \end{bmatrix}, \tag{5}$$

which is obtained by solving optimization problem in (6): [MS: Must be revised!]

$$\min_{\widehat{\boldsymbol{\theta}}} \quad J(\boldsymbol{e}; \widehat{\boldsymbol{\theta}}) := \tfrac{1}{2}\boldsymbol{e}^\top \boldsymbol{e} \tag{6a}$$

$$\text{subject to} \quad c_i^\theta(\widehat{\boldsymbol{\theta}}_i) := \tfrac{1}{2}\|\widehat{\boldsymbol{\theta}}_i\|^2 - \tfrac{1}{2}\bar{\theta}_i^2 \le 0, \ \forall i \in \mathcal{K} \tag{6b}$$

$$c_j^u(\widehat{\boldsymbol{\theta}}) \le 0, \quad \forall j \in \mathcal{I}, \tag{6c}$$

where

$$\mathcal{I} = \{C0, C1, \dots, Ck_c\} \cup \{F0, F1, \dots, Fk_f\},$$

c_i are the weight norm constraints (i.e. ensuring boundedness) and $c_{\widehat{\boldsymbol{\Phi}}}$ is the output norm constraint (i.e. ensuring input saturation limits $\bar{u}$).

Solving the optimization problem in (**??**) is equivalent to finding the saddle point of the Lagrangian function given in (7):

$$\mathcal{L}(\widehat{\boldsymbol{\theta}}, \boldsymbol{\lambda}) = \tfrac{1}{2}e^\top e + \sum_{i \in \mathcal{I}} \lambda_i c_i(\widehat{\boldsymbol{\theta}}) + \lambda_{\widehat{\boldsymbol{\Phi}}} c_{\widehat{\boldsymbol{\Phi}}}(\widehat{\boldsymbol{\theta}}), \tag{7}$$

where $\lambda_i, \lambda_{\widehat{\boldsymbol{\Phi}}} \ge 0$ are the Lagrange multipliers.

The weight update for each block $\widehat{\boldsymbol{\theta}}_i$ is proportional to the negative gradient of the Lagrangian: $\dot{\widehat{\boldsymbol{\theta}}}_i = -\alpha \nabla_{\widehat{\boldsymbol{\theta}}_i} \mathcal{L}$.

1) Objective Gradient: The objective gradient is approximated using the chain rule and the sensitivity approximation $\frac{\partial e}{\partial u} = \frac{\partial e}{\partial \widehat{\boldsymbol{\Phi}}} \approx \mathbf{I_n}$:

$$\nabla_{\widehat{\boldsymbol{\theta}}_i} \mathcal{J} = \left(\frac{\partial e}{\partial \widehat{\boldsymbol{\theta}}_i}\right)^T e \approx \left(\frac{\partial e}{\partial \widehat{\boldsymbol{\Phi}}} \frac{\partial \widehat{\boldsymbol{\Phi}}}{\partial \widehat{\boldsymbol{\theta}}_i}\right)^T e = \left(\frac{\partial \widehat{\boldsymbol{\Phi}}}{\partial \widehat{\boldsymbol{\theta}}_i}\right)^T e, \tag{8}$$

The gradients of the constraints are given by:

$$\begin{cases} \nabla_{\widehat{\boldsymbol{\theta}}_i} c_i = \widehat{\boldsymbol{\theta}}_i, & \forall i \in \mathcal{I} \\ \nabla_{\widehat{\boldsymbol{\theta}}_i} c_{\widehat{\boldsymbol{\Phi}}} = \left(\frac{\partial \widehat{\boldsymbol{\Phi}}}{\partial \widehat{\boldsymbol{\theta}}_i}\right)^T \widehat{\boldsymbol{\Phi}}, & \forall i \in \mathcal{I}, \end{cases} \tag{9}$$

The resulting gradient descent weight update rule for the primal variables is:

$$\dot{\widehat{\boldsymbol{\theta}}}_i = -\alpha \left[\left(\frac{\partial \widehat{\boldsymbol{\Phi}}}{\partial \widehat{\boldsymbol{\theta}}_i}\right)^T e + \lambda_i \widehat{\boldsymbol{\theta}}_i + \lambda_{\widehat{\boldsymbol{\Phi}}} \left(\frac{\partial \widehat{\boldsymbol{\Phi}}}{\partial \widehat{\boldsymbol{\theta}}_i}\right)^T \widehat{\boldsymbol{\Phi}} \right], \tag{10}$$

where $\alpha \in \mathbb{R}_{>0}$ is the learning rate.

The Lagrange multipliers are updated using projected gradient ascent:

$$\begin{cases} \dot{\lambda}_j = \beta_j c_j(\widehat{\boldsymbol{\theta}}), & \forall j \in \mathcal{I} \cup \{\widehat{\boldsymbol{\Phi}}\} \\ \lambda_j & = \max(\lambda_j, 0), \end{cases} \tag{11}$$

where $\beta_j \in \mathbb{R}_{>0}$ is the update rate for the corresponding multiplier.

IV. Stability Analysis

[MS: will be added later]

V. Numerical Validation

A. Validation Setup

In this section, the proposed controller is validated through numerical simulation of the 4 wheel steering (4WS) vehicle model [Rajamani, 2012] described as follows:

$$\begin{cases} \frac{\mathrm{d}}{\mathrm{d}t} v_y = \frac{1}{m}\left(F_f + F_r\right) - v_x r, \\ \frac{\mathrm{d}}{\mathrm{d}t} r = \frac{1}{I_z}\left(F_f l_f - F_r l_r\right). \end{cases} \tag{12}$$

The nomenclature is listed in Table I. The lateral tire forces F_f and F_r are the main sources of nonlinearity in the system, and F_f and F_r are modelled as:

$$F_i = F_{\max,i} \tanh\left(\frac{C_i}{F_{\max,i}} \alpha_i\right), \quad \forall i \in \{f, r\}, \tag{13}$$

where $F_{\max,i}$ are saturation limits and α_f and α_r are the slip angles for the front and rear tires, respectively. The slip angles are given by

$$\alpha_f = \delta_f - \frac{v_y + l_f r}{v_x}, \quad \alpha_r = \delta_r - \frac{v_y - l_r r}{v_x}. \tag{14}$$

In order to capture the lag between the steering commands and the actual steering angles, the actuator dynamics are modeled as:

$$\frac{\mathrm{d}}{\mathrm{d}t}\delta_i = \frac{1}{\tau}\left(\omega_i - \delta_i\right), \quad \forall i \in \{f, r\}, \tag{15}$$

where ω_f and ω_r denotes the front and rear steering commands, respectively, and $\tau > 0$ is the actuator time constant.

By defining the state as $\boldsymbol{x}_1 := (v_y, r)^\top$ and $\boldsymbol{x}_2 := (\delta_f, \delta_r)^\top$, and the control input as $\boldsymbol{u} := (\omega_f, \omega_r)^\top$, the system can be expressed in the form of (1). Additionally, only the output $\boldsymbol{y} = \boldsymbol{x}_1$ can be measured, and the state $\boldsymbol{x}_2$ is not measurable by the system definition.

[MS: Depending on the problem formulation, state can be position. (for now it is speed and time lag is added in control input intentionally to make the system second order) Tell me later how are you implementing controllers (what is the error for now? if the position is error as you wrote here, then the system

Symbol	Description	Unit
m	Mass of the vehicle	kg
I_z	Yaw moment of inertia of the vehicle	$\mathrm{kg\,m^2}$
v_x	Longitudinal velocity of the vehicle	$\mathrm{m\,s^{-1}}$
v_y	Lateral velocity of the vehicle	$\mathrm{m\,s^{-1}}$
r	Yaw rate of the vehicle	$\mathrm{rad\,s^{-1}}$
l_f, l_r	Distance from CG to front/rear axle	m
C_f, C_r	Cornering stiffness of front/rear tires	$\mathrm{N\,rad^{-1}}$
F_f, F_r	Lateral forces at front/rear tires	N
δ_f, δ_r	Front/rear steering angle input	rad

TABLE I: Nomenclature for the 4WS vehicle model.

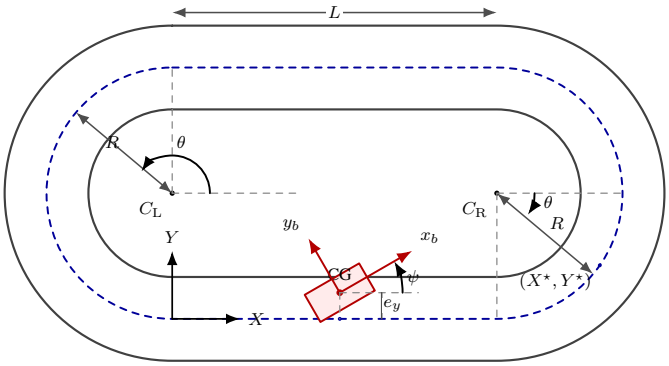

Fig. 2: Geometry of the stadium path and vehicle pose.

is actuall third order)] The vehicle parameters are selected as $m = 1463\,\mathrm{kg}$, $I_z = 1967.8\,\mathrm{kg\,m^2}$, $l_f = 1.2\,\mathrm{m}$, and $l_r = 1.6\,\mathrm{m}$. The longitudinal velocity is fixed at $v_x = 110\,\mathrm{km\,h^{-1}}$. The front and rear cornering stiffness values are allowed to vary within $C_i \in [78.4\,, 145.6\,\mathrm{kN\,rad^{-1}}]$ for $i \in \{f, r\}$, which corresponds to a $\pm 30\%$ variation about the nominal value $112\,\mathrm{kN\,rad^{-1}}$. [MS: where is Fmax, tau?]

The desired trajectory is designed as a closed stadium-shaped path composed of two straight segments and two semicircular arcs, as illustrated in Fig. 2. Let L and R denote the straight length and turning radius, respectively. [MS: I omitted many parts below. They look unnecessary. If you think they are necessary, please let me know.]

For comparative study, we consider the following three controllers as listed in Table II. [MS: Should be added more later]

[MS: Maybe below can be moved to our document]
The global position kinematics are given by (16):

$$\dot{X}_a = v_x \cos\psi - v_y \sin\psi, \quad \dot{Y}_a = v_x \sin\psi + v_y \cos\psi, \quad (16)$$

where (X_a, Y_a) denotes the vehicle position in the global frame, and ψ denotes the yaw angle. The yaw rate $\dot{\psi}$ is obtained by integrating the yaw acceleration $\ddot{\psi}$ in (12).

To generate the path-following reference at each simulation step, the current vehicle position (X_a, Y_a) is projected onto the corresponding path segment. For straight segments, the desired heading is:

$$\psi_{\mathrm{des}} = \begin{cases} 0, & \text{lower straight,} \\ \pi, & \text{upper straight,} \end{cases} \quad (17)$$

For the arc segments, the projected point is computed by

$$X^\star = x_c + R\cos\theta, \qquad Y^\star = y_c + R\sin\theta, \quad (18)$$

with

$$\theta = \mathrm{atan2}(Y_a - y_c, X_a - x_c), \quad (19)$$

The desired heading is then taken as the tangent direction of the arc,

$$\psi_{\mathrm{des}} = \theta + \frac{\pi}{2}, \quad (20)$$

The tracking errors are then defined by the lateral displacement from the projected point and the heading error.

TABLE II: Controllers for comparative study. [MS: Added for later]

	Description	Control Input
(C_1) [—]	Proposed controller	(??)
(C_2) [—]	Conventional feedback controller based on contraction theory [?]	(??) without NN, i.e., $\boldsymbol{\nu} = \mathbf{0}$
(C_3) [—]	Desired controller	$\boldsymbol{u}_d$

B. Validation Results

The validation results are presented in terms of tracking errors, steering commands, and the global vehicle trajectory. In particular, the lateral and heading errors are used to assess whether the proposed 1D CNN–MLP controller can regulate the vehicle toward the desired path while adapting its parameters online. The front and rear steering-rate commands illustrate the control effort generated by the end-to-end policy, and the global XY trajectory provides a direct geometric comparison between the desired path and the actual motion.

In the final manuscript, this subsection should be organized around the simulation plots produced by the current MATLAB implementation. A compact presentation consistent with the reference paper would first discuss the validation conditions, then summarize the tracking behavior in the time domain, and finally comment on the overall path-following accuracy and adaptation behavior. Root-mean-square tracking errors and selected controller settings can also be reported in a table to make the comparison more concise.

VI. Conclusion

Appendix

A. Analytic Jacobian of NN

[MS: here, it says that the detail of the Jacobian calculation is in Appendix B (in previous version). Can you add here?]

The adaptation law in (10) requires weights Jacobians and are given by:

1) FCL Jacobians: The Jacobians of $\widehat{\mathbf{\Phi}}$ with respect to the weights of FCL are derived recursively as the follows:

$$\frac{\partial \widehat{\mathbf{\Phi}}}{\partial \widehat{\boldsymbol{\theta}}_{F j_f}} = \begin{cases} \left(\prod_{l=1}^{k_f} V_l^\top \phi_l^{F'}\right)\left(I_{l_0} \otimes C(\Phi_{k_c}^C)\right), & j_f = 0 \\ \left(\prod_{l=j_f+1}^{k_f} V_l^\top \phi_l^{F'}\right)\left(I_{l_{j_f}} \otimes \Phi_{j_f-1}^{F^\top}\right), & j_f \in [1, \dots, k_f], \end{cases} \tag{21}$$

where $\phi_{j_f}^{F'} \triangleq \frac{\partial \phi_{j_f}^F(x)}{\partial x}$ is the Jacobian of the activation function with respect to some vector x.

2) CVL Jacobians: The Jacobians of $\widehat{\mathbf{\Phi}}$ with respect to the weights of CVLs (filters Ω_{j_c} and biases B_{j_c}) are given as follows:

$$\frac{\partial \widehat{\mathbf{\Phi}}_i}{\partial W_{j_c}^{l_k}} = \sum_{l_i=1}^{n_{j_c+1}} \left(\frac{\partial \widehat{\mathbf{\Phi}}_i}{\partial \mathbf{\Phi}_{j_c}^C(l_i, l_k)} \operatorname{row}(l_b : l_e)(\mathbf{\Phi}_{j_c}^C) \right), \tag{22}$$

$$\frac{\partial \widehat{\mathbf{\Phi}}_i}{\partial B_{j_c}(l_k)} = \sum_{l_i=1}^{n_{j_c+1}} \left(\frac{\partial \widehat{\mathbf{\Phi}}_i}{\partial \mathbf{\Phi}_{j_c}^C(l_i, l_k)} \cdot 1 \right), \tag{23}$$

with $l_b \triangleq l_i$, $l_e \triangleq l_i + p_{j_c} - 1$, $j_c \in [0, \dots, k_c]$, $l_k \in [1, \dots, q_{j_c}]$

where Φ_i denote the i-th output of $\widehat{\mathbf{\Phi}}$, $\mathbf{\Phi}_{j_c}^C \triangleq \mathbf{\Phi}_{j_c}^C(\Phi_{j_c-1}^C)$ for $j_c \in [1, \dots, k_c]$, $\Phi_0^C \triangleq \phi_0^C(X)$, and $\partial \widehat{\mathbf{\Phi}}_i / \partial \mathbf{\Phi}_{j_c}^C$ denotes the backpropagated gradient of $\widehat{\mathbf{\Phi}}_i$ with respect to $\mathbf{\Phi}_{j_c}^C$ (obtained using the back-propagation method, detailed in Appendix B).

REFERENCES

[IEE,]

[Bernstein and Bernstein, 2009] Bernstein, D. S. and Bernstein, D. S. (2009). *Matrix Mathematics: Theory, Facts, and Formulas (Second Edition)*. Princeton University Press, Princeton.

[Esfandiari et al., 2022] Esfandiari, K., Abdollahi, F., and Talebi, H. A. (2022). *Neural network-based adaptive control of uncertain nonlinear systems*. Springer.

[Griffis et al., 2023] Griffis, E. J., Patil, O. S., Bell, Z. I., and Dixon, W. E. (2023). Lyapunov-based long short-term memory (Lb-LSTM) neural network-based control. *IEEE Control Systems Letters*.

[Hart et al., 2023] Hart, R. G., Griffis, E. J., Patil, O. S., and Dixon, W. E. (2023). Lyapunov-based physics-informed long short-term memory (LSTM) neural network-based adaptive control. *IEEE Control Systems Letters*.

[Jhung et al., 2018] Jhung, J., Bae, I., Moon, J., Kim, T., Kim, J., and Kim, S. (2018). End-to-end steering controller with cnn-based closed-loop feedback for autonomous vehicles. In *2018 IEEE intelligent vehicles symposium (IV)*, pages 617–622. IEEE.

[Liu and Dong, 2017] Liu, X. and Dong, Y. (2017). A convolutional neural networks approach to devise controller. In *MATEC Web of Conferences*, volume 139, page 00168. EDP Sciences.

[Nobahari and Seifouripour, 2019] Nobahari, H. and Seifouripour, Y. (2019). A nonlinear controller based on the convolutional neural networks. In *2019 7th International Conference on Robotics and Mechatronics (ICRoM)*, pages 362–367. IEEE.

[Patil et al., 2021] Patil, O. S., Le, D. M., Greene, M. L., and Dixon, W. E. (2021). Lyapunov-derived control and adaptive update laws for inner and outer layer weights of a deep neural network. *IEEE Control Systems Letters*, 6:1855–1860.

[Rajamani, 2012] Rajamani, R. (2012). *Vehicle dynamics and control*. Mechanical Engineering Series. Springer, New York, NY, 2 edition.

[Rausch et al., 2017] Rausch, V., Hansen, A., Solowjow, E., Liu, C., Kreuzer, E., and Hedrick, J. K. (2017). Learning a deep neural net policy for end-to-end control of autonomous vehicles. In *2017 American Control Conference (ACC)*, pages 4914–4919. IEEE.

[Talebi et al., 2010] Talebi, H., Abdollahi, F., Patel, R., and Khorasani, K. (2010). *Neural Network-Based State Estimation of Nonlinear Systems: Application to Fault Detection and Isolation*. Lecture Notes in Control and Information Sciences. Springer New York.

Myeongseok Ryu received the B.S. degree in Mechanical Engineering from Incheon National University, South Korea, in 2023, and the M.S. degree in Mechanical Engineering from GIST, South Korea, in 2025. He is currently pursuing the Ph.D. degree in the Cho Chun Shik Graduate School of Mobility at Korea Advanced Institute of Science and Technology (KAIST), South Korea. His research interests include neuro-adaptive control, online optimization, and contraction theory.

Naol Samuel Erega received his B.S. degree in Aerospace Engineering and Electrical Engineering from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2026. He is currently pursuing the M.S. degree at the Cho Chun Shik Graduate School of Mobility, KAIST. His research interests include neural network-based control, system estimations, and unmanned ground vehicle(UGV) dynamics.

Kyunghwan Choi received his B.S., M.S., and Ph.D. degrees in Mechanical Engineering from KAIST, Daejeon, South Korea, in 2014, 2016, and 2020, respectively. Following his Ph.D., he worked at the Center for Eco-Friendly & Smart Vehicles at KAIST as a Postdoctoral Fellow and later as a Research Assistant Professor. In 2022, he joined GIST as an Assistant Professor in the Department of Mechanical and Robotics Engineering. Currently, he serves as an Assistant Professor at the Cho Chun Shik Graduate School of Mobility at KAIST, where he is also the Director of the Mobility Intelligence and Control Laboratory. His research focuses on optimal and learning-based control for connected, automated, and electrified vehicles (CAEVs).